TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant using LangGraph & HITL

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines:

- Information retrieval from external data sources
- Text generation using a Large Language Model (LLM)

Instead of relying only on pre-trained knowledge, the system retrieves relevant information from a document and generates accurate responses.

Why RAG is Needed

- LLMs may generate incorrect or outdated answers
- RAG ensures **context-aware and document-based responses**
- Improves reliability and reduces hallucination

Use Case Overview

This project implements a **Customer Support Assistant** that:

- Uses a PDF (Sales & Distribution Management) as knowledge base
- Answers user queries based only on document content
- Escalates queries to humans when information is unavailable

➢ **Flow:**

1. User inputs query
2. Query converted to embedding
3. Relevant chunks retrieved from ChromaDB
4. Context passed to LLM
5. LLM generates answer
6. System evaluates response
7. If low confidence → HITL triggered

2. System Architecture Explanation

Document Ingestion

- Loads PDF using PyPDFLoader
- Extracts raw text from document

---

2. Chunking

- Splits document into smaller chunks
- Uses:
  - Chunk size: 500
  - Overlap: 100

Ensures better retrieval accuracy

---

3. Embeddings

- Converts text into numerical vectors
- Uses **HuggingFace Embeddings (all-MiniLM-L6-v2)**

---

4. Vector Database

- Uses **ChromaDB**
- Stores embeddings for fast similarity search

---

5. Retrieval System

- Performs similarity search based on query
- Returns top relevant chunks

---

6. LLM Layer

- Uses **Flan-T5 (HuggingFace)**

- Generates response using retrieved context

---

7. Workflow (Control Logic)

- Input → Retrieval → Processing → Output
- Handles decision making

---

8. HITL (Human-in-the-Loop)

- Triggered when:
  - No relevant data found
  - Low confidence response
- Allows human intervention

9.Design Decisions

Chunk Size Selection

- Size: 500
- Reason:
  - Too small → loss of context
  - Too large → inefficient retrieval

Embedding Strategy

- Model: all-MiniLM-L6-v2
- Reason:
  - Lightweight
  - Fast
  - Good semantic similarity

Retrieval Approach

- Top-K similarity search (k=3)

- Ensures most relevant chunks are selected

Prompt Design

- LLM instructed to:
  - Use only provided context
  - Avoid hallucination
  - Return “I don’t know” if answer missing

➢ **Node Responsibilities**

- Retrieval Node → fetch relevant data
- Processing Node → generate answer
- Output Node → return response

➢ **State Transition**

State includes:

- Query
- Retrieved context
- Generated answer

Testing Strategy

➢ **Approach**

- Tested with:
  - Basic queries
  - Complex queries
  - Out-of-scope queries

➢ **Sample Queries**

- What is sales management?

- Explain AIDA model
- What is sales forecasting?
- What is AI in marketing? (HITL test)

4.Future Enhancements

- Multi-document support
- Feedback-based learning
- Memory integration
- Web UI (Streamlit)
- Real-time deployment

➢ **Conclusion**

This project successfully demonstrates:

- RAG-based system design
- Context-aware response generation
- Workflow-based control logic
- Human-in-the-Loop integration

It provides a scalable and reliable solution for customer support applications.